

AP(02) - Parte A e B**1s/UFSM****Período: 12-19/08/24****Acadêmico: André Gonçalves da Silva****Matrícula: 202210071****Curso/Turma: 123/11**

As questões da Avaliação Parcial 02 (AP(02)) tem por finalidade a aplicação direta dos aspectos teóricos e práticos dos métodos numéricos abordados na disciplina (Integração Numérica, Ajuste Funcional e Resolução de EDO(s) e recursos de programação em linguagem PYTHON 3 empregados nas respectivas implementações e cálculo numérico). A produção do arquivo-resposta seguirá os moldes da produção dos arquivos dos enunciados, nos formatos IPYNB e PDF e ambos deverão ser entregues via TAREFA aberta no MOODLE para a AP(02) de 19/08/2024.

Observa-se que a Parte A dessa avaliação substitui as mini-avaliações QUESTÃO (01), QUESTÃO (02) e QUESTÃO(03), substituição decorrente a excepcionalidade da alteração do calendário acadêmico de 2024.

OBSERVAÇÕES

- Salienta-se que observância da nomeação dos arquivos IPYNB e PDF (ver abaixo) e informações do cabeçalho (preencher devidamente) serão partes constituinte do escore (divisões sucessivas por 2):
 - 24_1s_MTM1013T11_AP02_AeB_NomeSobrenome.ipynb;
 - 24_1s_MTM1013T11_AP02_AeB_NomeSobrenome.pdf;
- Os algoritmos que devem ser empregados para as implementações em linguagem PYTHON 3 e *testes de mesas* são os algoritmos dados pelo material didático disponibilizado nos repositórios [MEGA](#) e [DRIVE](#) e não os sendo as respostas serão desconsideradas. Ressalta-se que os algoritmos do Material Didático são adequações dos algoritmos apresentados em Campos Filho (2018)[⁰], os quais podem ser utilizados para a implementação ou da Burden e Fayres (2016)[¹];

- A elaboração das questões buscará caracterizações personalizadas para cada aluno, fato que resultará em um diversificado conjunto de situações-testes para a exploração dos aspectos teóricos e práticos da disciplina em detrimento, eventualmente, da aplicabilidade do tema abordado na questão.

[0]: Filho, Frederico Ferreira C. Algoritmos Numéricos - Uma Abordagem Moderna de Cálculo Numérico, 3ª edição. Disponível em: [Minha Biblioteca](#), Grupo GEN, 2018. Ainda, há exemplares na BSCCNE.

[1]: Burden, Richard, L. et al. Análise Numérica - Tradução da 10ª edição norte-americana. Disponível em: [Minha Biblioteca](#), (3rd edição). Cengage Learning Brasil, 2016.

Obtenção de dados para elaboração das questões.

Ao considerar o meu número de matrícula na UFSM, 3332882, monta-se a tabela

d1	d2	d3	d4	d5	d6	d7	d8	d9	d10
0	0	0	3	3	3	2	8	8	2

e sobre essas informações, colocam-se as seguintes elaborações:

(I) Dedução dos parâmetros.

- $a = d5 = 3$ - caso $a = 0$, colocar $a = +1$;
- $b = d6 = 3$ - caso $b = 0$ ou $b = a$, colocar $b = +1$;
- $c = (d1 + d2 + d3 + d4) = (0 + 0 + 0 + 3) = 3$;
- $d = (d7 + d8 + d9 + d10) = (2 + 8 + 8 + 2) = 20 \xrightarrow{d \geq 10} d = (2 + 0) = 2$;
- $Dados = [a, b, c, d] = [3, 1, 3, 2]$;
- $r_0 = \min\{a, b, c, d\} = 1$;

(II) Sabe-se do estudo de fluidos que a quantidade de fluido por unidade de tempo $[(u. c.)^3 / (u. t.)]$ que atravessa esta seção uma seção circular de raio $r_0 = a$ é dado por:

$$Q = \frac{V}{t};$$

O volume pode ser descrito $V = Ad$, com A a área da seção transversal circular e d a profundidade desse quantidade que se quer medir (ver ilustração)[²], logo

$$Q = \frac{V}{t} = \frac{Ad}{t} = A \frac{d}{t} = Av_{fluido},$$

com $v_{fluido} = d/t$ a velocidade do fluido;

Sobre essa análise qualitativa do processo de medição da quantidade de fluido que passa pela seção transversal circular, pode-se montar formulações mais precisas para a quantificação dessa quantidade Q , como segue:

- Da disciplina de Cálculo B (ou II) problemas associados à quantificação do fluxo de um fluido através de uma superfície S é dada pela integral de superfície:

$$\begin{aligned} \text{FLUXO} &= \iint_S \mathbf{v}(x, y, z) \cdot \mathbf{n} dS = \iint_{D_{xz}} (v_x, v_y, v_z) \cdot \overbrace{\frac{\mathbf{N}}{\|\mathbf{N}\|}}^{\mathbf{n}} \underbrace{\|\mathbf{N}\|}_{dS} dA, \\ &= \iint_{D_{xz}} (v_x, v_y, v_z) \cdot (n_x, n_y, n_z) dA = \iint_{D_{xz}} (v_x n_x + v_y n_y + v_z n_z) dA \end{aligned}$$

cuja resolução dará a quantidade de fluido que passa pela seção por unidade de tempo;

- Considere o campo de velocidades do fluido dado por $\mathbf{v}(x, y, z) = \left(0, b\left(1 - \frac{\sqrt{x^2+z^2}}{c}\right)^{1/7}, 0\right)$, com o centro do tubo de raio r_0 alinhado com o vetor normal $\mathbf{n} = (n_x, n_y, n_z) = (0, 1, 0) = \mathbf{j} = \frac{\mathbf{N}}{\|\mathbf{N}\|}$ a superfície de corte seccional descrita acima;
- Devido a simetria circular da seção transversal, emprega-se coordenadas polares para a descrição do campo de velocidades e no cálculo da quantidade de fluxo, ie,

$$\blacksquare \mathbf{v}(x, y, z) = \left(0, b\left(1 - \frac{\sqrt{x^2+z^2}}{c}\right)^{1/7}, 0\right) \equiv \left(0, b\left(1 - \frac{r}{c}\right)^{1/7}, 0\right) \Rightarrow v_y(r, \theta) = v(r) = b\left(1 - \frac{r}{r_0}\right)^{1/7};$$

- Conforme dados acima, segue que

$$\begin{aligned}
Q &= \iint_{D_{xz}} v_y(x, z) dA, \quad D_{xz} := \{(x, z) \in \mathbb{R}^2 \mid 0 \leq x^2 + z^2 \leq r_0\} \\
&\equiv \iint_{D_{r\theta}} v_y(r, \theta) dA, \quad D_{r\theta} := \{(r, \theta) \in \mathbb{R}^2 \mid 0 < r < r_0 \wedge 0 \leq \theta < 2\pi\} \quad (x = r \cos \theta, z = r \sin \theta) . \\
&= \int_0^{r_0} \int_0^{2\pi} v_y(r, \theta) r dr d\theta = \int_0^{2\pi} d\theta \int_0^{r_0} r v_y(r, \theta) dr = 2\pi \int_0^{r_0} r v_y(r, \theta) dr = 2\pi \int_0^{r_0} r v(r) dr
\end{aligned}$$

- Os itens acima, com a lista de de Dados aplicáveis:

- $\mathbf{v}(r, y, \theta) = \left(0, 1\left(1 - \frac{r}{r_0}\right)^{1/7}, 0\right) = \left(0, \left(1 - \frac{r}{1}\right)^{1/7}, 0\right) = \left(0, (1 - r)^{1/7}, 0\right);$
- $D_{xz} := \{(x, z) \in \mathbb{R}^2 \mid 0 \leq x^2 + z^2 \leq r_0\} \equiv D_{xz} := \{(x, z) \in \mathbb{R}^2 \mid 0 \leq x^2 + z^2 \leq 1\};$

(III) Gere as tabelas, com a seguinte caracterização para a função $v(r)$ e o integrando $rv(r)$:

- (A) Os sete primeiros pontos igualmente afastados para o intervalo $[0; 0, 8r_0]$;
- (B) Os cinco últimos pontos igualmente afastados para o intervalo $[0, 8r_0; r_0]$;

E a partir destas listas montadas, plotar estes pares ordenados e as funções geratrizes via biblioteca MATPLOTLIB;

Obs.: Os dados obtidos formarão uma tabela no formato:

r	0	...	$0, 8r_0$	...	r_0
$v(r)$	b	...	$v(0, 8r_0)$	...	$v(r_0)$
$rv(r)$	0	...	$(0, 8r_0)v(0, 8r_0)$	...	$r_0v(r_0)$

E os gráficos via WolframAlpha, para os dados acima:

- [Gráfico de \$v\(r\)\$](#) ;
- [Gráfico do integrando \$rv\(r\)\$](#) ;

(IV) A partir de (I), montar o PVI:
$$\begin{cases} ay' = b + \frac{cy}{dx}, x \in [1, 2] \\ y(1) = 2 \text{ (} h = 0,05 \text{)} \end{cases} \Leftrightarrow \begin{cases} 3y' = 1 + \frac{3y}{2x} \\ y(1) = 2 \end{cases}$$

(V) Para gráficos pode-se usar as páginas tutoriais: [MATPLOLIB](#), [SYMPY](#) ou [SYMPY\(2\)](#).

[2]: KHAN ACADEMY. O que é taxa de fluxo de volume? Disponível em: <<https://pt.khanacademy.org/science/physics/fluids/fluid-dynamics/a/what-is-volume-flow-rate>>. Acesso em: 12/08/2024.

QUESTÕES - PARTE A

1) (1,0 ponto) Faça o que se pede, a partir das instruções acima:

1.1) (0,2 ponto [Dados]) Informe sua matrícula como uma lista ($matricula = [d1, d2, d3, d4, d5, d6, d7, d8, d9, d10]$), bem como a lista de parâmetros Dados;

1.2) (0,2 ponto [Dados]) Monte a integral a ser resolvida em r (ver (II)) e destaque a função v ;

1.3) (0,3 ponto [Dados]) Informe as tabelas em (III) em forma de lista e plote os diagramas de dispersão associados;

1.4) (0,3 ponto [Dados]) Informe o PVI em (IV) para os parâmetros associados à sua matrícula;

2) (1,0 ponto) Sobre o Item (II) e a Questão 1.2, faça o que se pede:

2.1) (0,25 ponto [Dados]) Plote o campo de direções tridimensionais no domínio D_{xz} , bem como a superfície $v_y(r, \theta) = v(r) = b\left(1 - \frac{r}{c}\right)^{1/7}$ definida neses domínio;

2.2) (0,25 ponto [Dados]) Plote as funções $v(r)$ e $rv(r)$ confrontando com os dados tabelados no intervalo completo;

2.3) (0,5 ponto [Dados]) Informe o resultado da integral em r com 9 casas de precisão e informe Q ;

3) (1,5 pontos) A partir dos dados em 1.3) e (III), faça o que se pede:

3.1) (0,5 ponto) Utilize o MMQ para encontrar os ajustes funcionais nos respectivos intervalos dados pelas tabelas acima;

(Sugestão: Utilize uma ajuste quadrático para o primeiro subintervalo e um ajuste exponencial para o segundo subintervalo.)

3.2) (0,5 ponto) Plote essas representações confrontando com os Diagramas de Dispersão dados;

3.3) (0,5 ponto) Com as informações obtidas em 3.1, calcule as integrais nos respectivos subintervalos e informe Q ;

4) (1,0 ponto) Resolva o PVI (1.4) (ver (IV)) pelo Método de Heun e apresente uma tabela, de 10 em 10 valores das soluções, que confronte a solução numérica com a solução analítica, bem como o erro real associado. Faça um gráfico comparando a solução analítica *versus* a solução numérica e as linhas de fluxo associadas;

QUESTÕES - PARTE B

OBSERVAÇÃO: Colocar os documentos IPYNB e PDF da resolução no espaço TAREFA do MOODLE aberta, caso queira entregar o arquivo-resposta com esses recursos computacionais. Caso se resolva via Google COLAB, colocar link compartilhado sem restrições de acesso e também o arquivo PDF associado.

```
In [ ]: import numpy as np
import math
from math import trunc
import matplotlib.pyplot as plt
#se tiver dando problema aqui é só comentar essa linha seguinte
from IPython.display import Markdown as md
```

```
In [ ]: d1 = 0
d2 = 2
d3 = 0
d4 = 2
d5 = 2
d6 = 1
d7 = 0
d8 = 0
d9 = 7
d10 = 1
```

```

a = d5
if a == 0:
    a = 1

b = d6
#se a == b == 1 como que fica??
if b == 0 or b == a:
    b = 1

c = d1 + d2 + d3 + d4
d = d7 + d8 + d9 + d10

r0 = np.min([a, b, c, d])

```

Resposta da 1.1

```

In [ ]: print("Matrícula = %s"%([d1, d2, d3, d4, d5, d6, d7, d8, d9, d10]))
        print("Dados = %s"%([a, b, c, d]))
        print("r_0 = %s"%r0)

```

```

Matrícula = [0, 2, 0, 2, 2, 1, 0, 0, 7, 1]
Dados = [2, 1, 4, 8]
r_0 = 1

```

Resposta da 1.2

A integral a ser resolvida é

$$Q = 2\pi \int_0^{r_0} rv(r)dr, \quad (1)$$

onde

$$v(r, y, \theta) = v_y(r) = b \left(1 - \frac{r}{r_0}\right)^{1/7}. \quad (2)$$

No meu caso:

$$Q = 2\pi \int_0^1 r v(r) dr, \quad (3)$$

$$v(r) = (1 - r)^{1/7}. \quad (4)$$

```
In [ ]: #tem como fazer as equações em latex com as variaveis do python
Qstring = (r"\begin{equation} Q = 2\pi \int_0^1 r v(r) dr, \end{equation}")
vstring = (r"\begin{equation} v(r) = {d} \bigg(1 - \frac{r}{d} \bigg)^{1/7}. \end{equation}")
md(Qstring%(r0) + "\n\n" + vstring%(b,r0))
```

Out[]:

$$Q = 2\pi \int_0^1 r v(r) dr, \quad (5)$$

$$v(r) = 1 \left(1 - \frac{r}{1}\right)^{1/7}. \quad (6)$$

Resposta da 1.3

```
In [ ]: v = lambda r: b*(1 - r/r0)**(1/7)
r1 = np.linspace(0, 0.8*r0, 7)
r2 = np.linspace(0.8*r0, r0, 5)

vrvals1 = v(r1)
vrvals2 = v(r2)

rvrvals1 = r1*vrvals1
rvrvals2 = r2*vrvals2

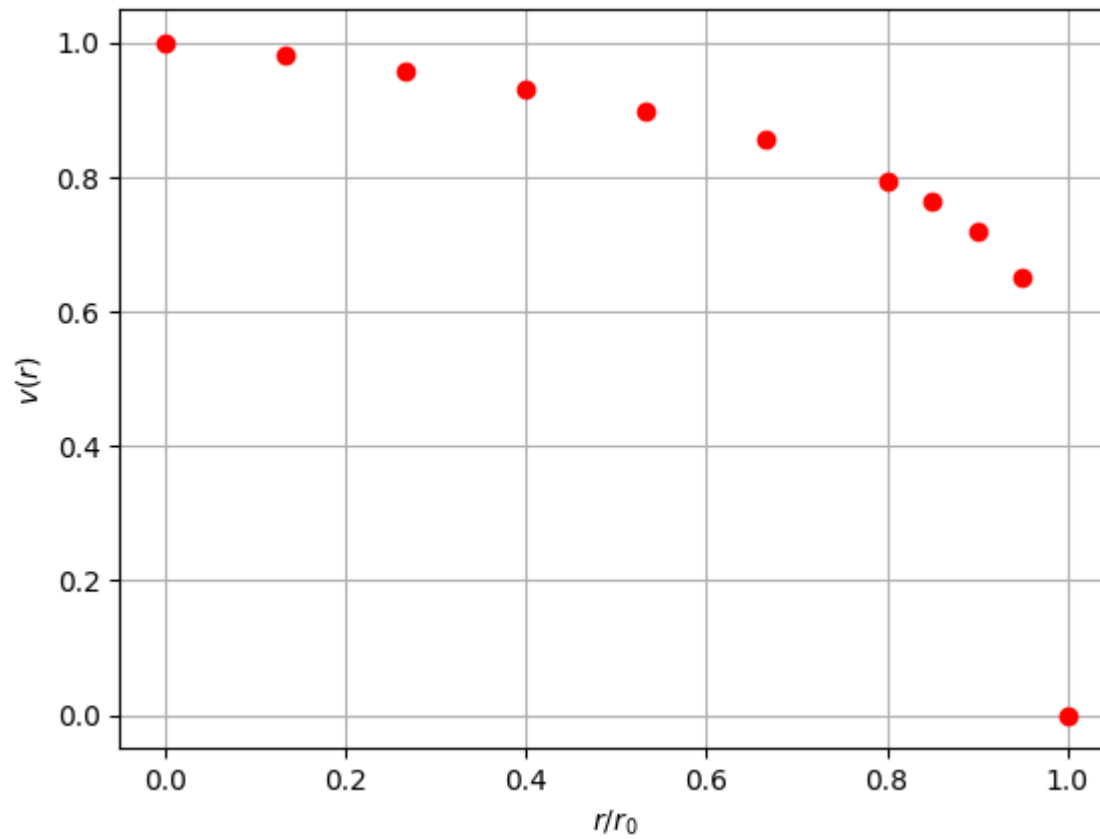
#começa no 1 os segundos porque não quero o ponto com r = 0.8*r0 sendo repetido
rvals = np.concatenate((r1, r2[1:]))
vrvals = np.concatenate((vrvals1, vrvals2[1:]))
rvrvals = np.concatenate((rvrvals1, rvrvals2[1:]))
```

```
float_formatter = "{:.3f}".format
np.set_printoptions(formatter={'float_kind':float_formatter})
print("Tabelas da (III) em forma de lista (1a linha: valores de r/r0, 2a linha: valores de v(r), 3a linha: valores
print(rvals)
print(vrvals)
print(rvrvals)
```

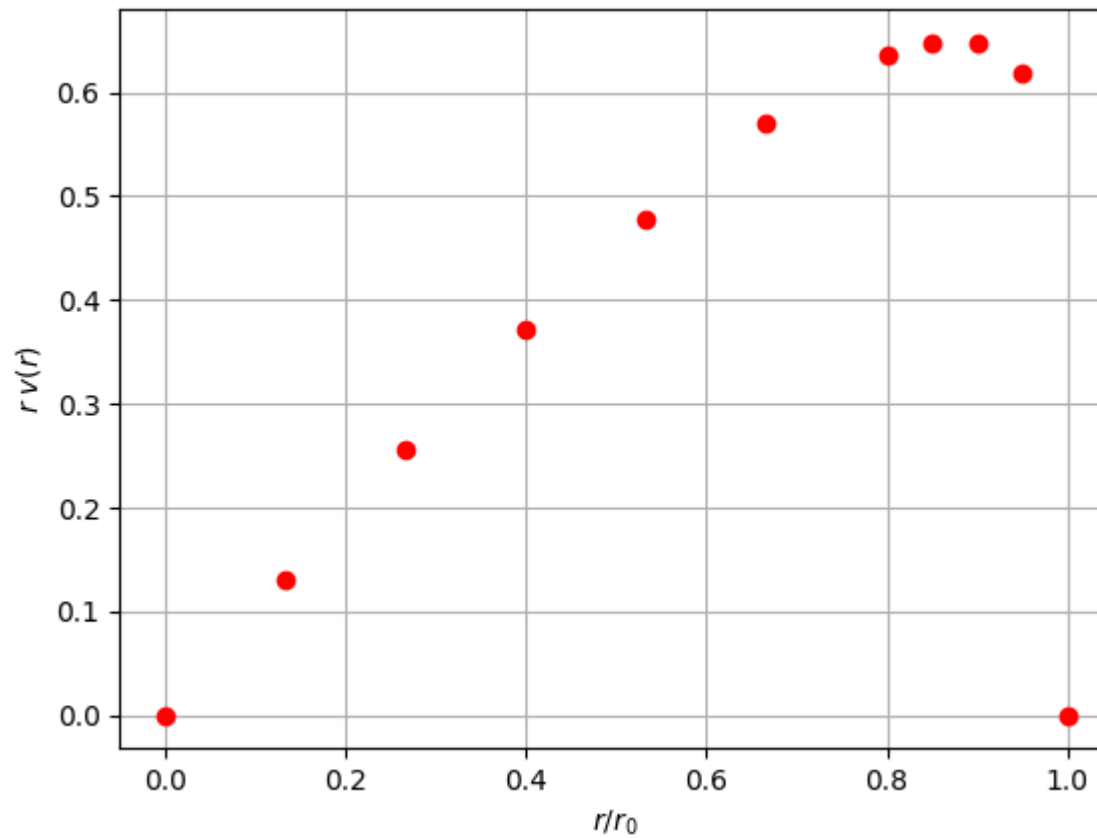
Tabelas da (III) em forma de lista (1a linha: valores de r/r_0 , 2a linha: valores de $v(r)$, 3a linha: valores de $r*v(r)$):

```
[0.000 0.133 0.267 0.400 0.533 0.667 0.800 0.850 0.900 0.950 1.000]
[1.000 0.980 0.957 0.930 0.897 0.855 0.795 0.763 0.720 0.652 0.000]
[0.000 0.131 0.255 0.372 0.478 0.570 0.636 0.648 0.648 0.619 0.000]
```

```
In [ ]: plt.plot(rvals, vrvals, "ro", label="$v(r)$")
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
plt.grid()
plt.show()
```



```
In [ ]: plt.plot(rvals, rvrvals, "ro", label="r v(r)")
plt.xlabel("$r/r_0$")
plt.ylabel("$r \setminus; v(r)$")
plt.grid()
plt.show()
```



Vemos que é na realidade um caso de escoamento de um fluido viscoso no tubo, perto das fronteiras do tubo o fluido tem menos velocidade.

Resposta da 1.4

O PVI é dado por:

$$\begin{cases} ay' = b + \frac{cy}{dx}, x \in [1, 2] \\ y(1) = 2, (h = 0,05) \end{cases}, \quad (7)$$

com os meus dados:

$$\begin{cases} 2y' = 1 + \frac{4y}{8x}, x \in [1, 2] \\ y(1) = 2, (h = 0,05) \end{cases} \Leftrightarrow \begin{cases} 2y' = 1 + \frac{y}{2x} \\ y(1) = 2 \end{cases}. \quad (8)$$

Resposta da 2.1

```
In [ ]: rparam = np.linspace(0, r0, 100)
thetaparam = np.linspace(0, 2*np.pi, 100)
rmesh, thetamesh = np.meshgrid(rparam, thetaparam)

Dx = rmesh*np.cos(thetamesh)
Dy = 0*rmesh
Dz = rmesh*np.sin(thetamesh)

rparamvec = np.linspace(0, r0, 8)
thetaparamvec = np.linspace(0, 2*np.pi, 3)

rmeshvec, thetameshvec = np.meshgrid(rparamvec, thetaparamvec)

x = rmeshvec*np.cos(thetameshvec)
y = 0*rmeshvec
z = rmeshvec*np.sin(thetameshvec)
vxvec = 0*rmeshvec
vyvec = (b*(1 - rmeshvec/r0)**(1/7))
vzvec = 0*rmeshvec

fig = plt.figure(figsize=(8, 6), layout="constrained")
ax = fig.add_subplot(111, projection='3d')

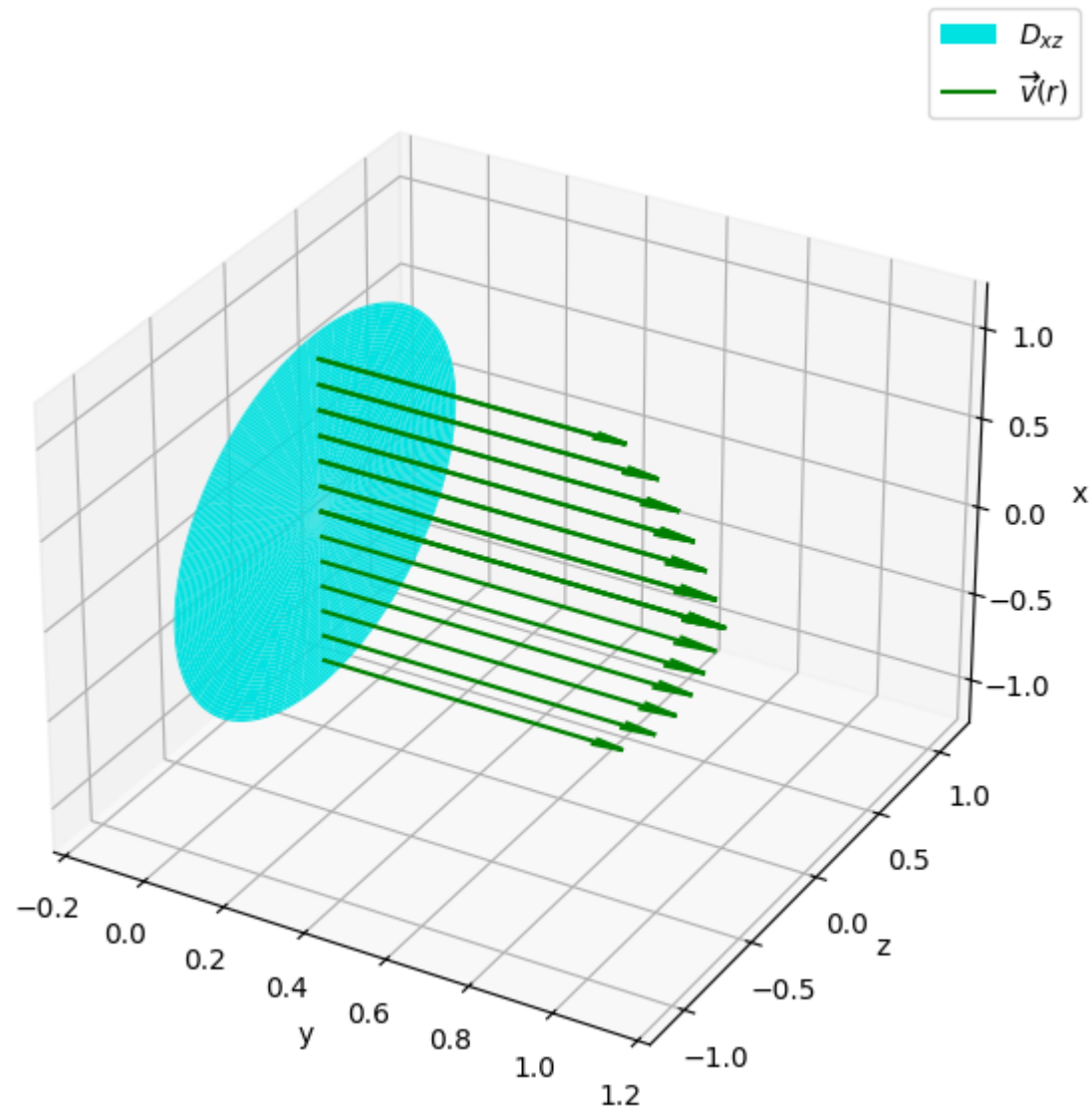
ax.plot_surface(Dx, Dy, Dz, alpha=1.0, color="cyan")
ax.quiver(x, y, z, vxvec, vyvec, vzvec, color="green", length=1.0, arrow_length_ratio=0.1, normalize=False)

ax.view_init(vertical_axis="x")
ax.set_box_aspect(aspect=None, zoom=0.9)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
ax.set_xlim3d(-1.2, r0 + 0.2)
```

```

ax.set_ylim3d(-0.2, r0 + 0.2)
ax.set_zlim3d(-1.2, r0 + 0.2)
ax.legend(["$D_{xz}$", "$\overrightarrow{v}(r)$"])
plt.show()

```



```

In [ ]: rparam = np.linspace(0, r0, 100)
        thetaram = np.linspace(0, 2*np.pi, 100)
        rmesh, thetamesh = np.meshgrid(rparam, thetaram)

        Dx = rmesh*np.cos(thetamesh)
        Dy = 0*rmesh
        Dz = rmesh*np.sin(thetamesh)

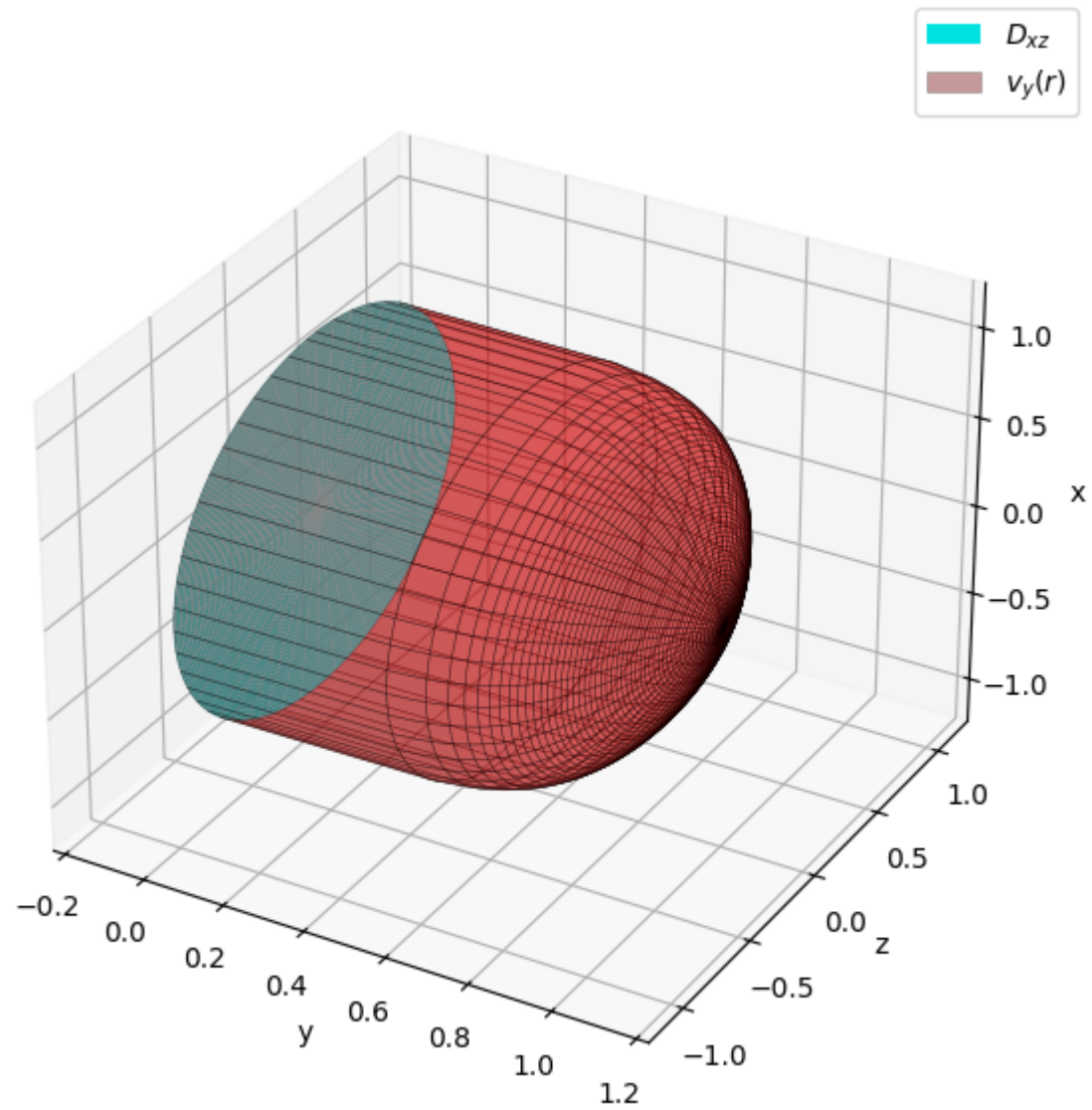
        vx = rmesh*np.cos(thetamesh)
        vy = (b*(1 - rmesh/r0)**(1/7))
        vz = rmesh*np.sin(thetamesh)

        fig = plt.figure(figsize=(8, 6), layout="constrained")
        ax = fig.add_subplot(111, projection='3d')

        ax.plot_surface(Dx, Dy, Dz, alpha=1.0, color="cyan")
        ax.plot_surface(vx, vy, vz, alpha=0.4, color="red", edgecolor="Black", lw=0.4)

        ax.view_init(vertical_axis="x")
        ax.set_box_aspect(aspect=None, zoom=0.9)
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.set_zlabel("z")
        ax.set_xlim3d(-1.2, r0 + 0.2)
        ax.set_ylim3d(-0.2, r0 + 0.2)
        ax.set_zlim3d(-1.2, r0 + 0.2)
        ax.legend([" $D_{xz}$ ", " $v_y(r)$ "])
        plt.show()

```



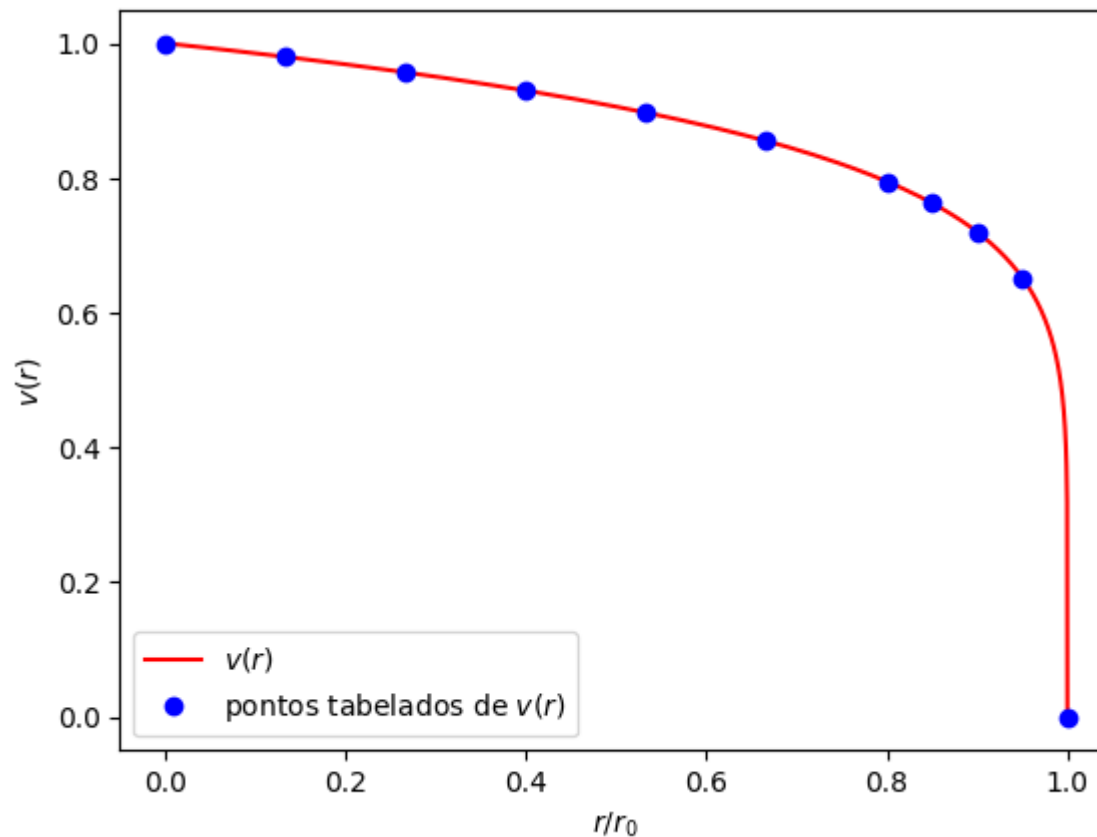
Resposta da 2.2

```

In [ ]: #não é exatamente um plot smooth, daria para fazer algo talvez mais suave com sympy
rsmooth1 = np.linspace(0, 0.9*r0, 100)
rsmooth2 = np.linspace(0.9*r0, r0, 300)
rsmooth = np.concatenate((rsmooth1, rsmooth2[1:]))
vsmooth = v(rsmooth)

plt.plot(rsmooth, vsmooth, "r-", label="$v(r)$")
plt.plot(rvals, vrvals, "bo")
plt.legend(["$v(r)$", "pontos tabelados de $v(r)$"])
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
plt.show()

```



```

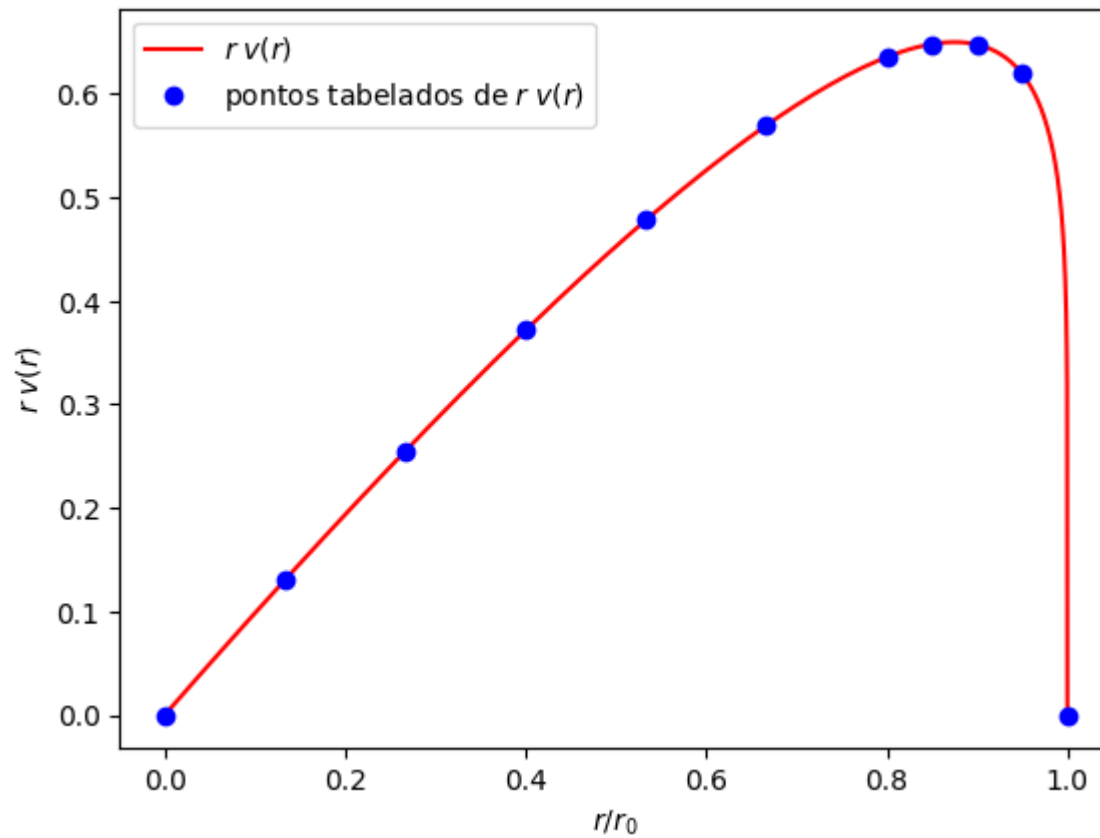
In [ ]: rvsmooth = rsmooth*vsmooth

```

```

plt.plot(rsmooth, rvsmooth, "r-", label="$r \; v(r)$")
plt.plot(rvals, rvvals, "bo")
plt.legend(["$r \; v(r)$", "pontos tabelados de $r \; v(r)$"])
plt.xlabel("$r/r_0$")
plt.ylabel("$r \; v(r)$")
plt.show()

```



Retomando a integral:

$$Q = 2\pi \int_0^1 rv(r)dr, \tag{9}$$

$$v(r) = (1 - r)^{1/7}. \tag{10}$$

```
In [ ]: #código de newton cotes

def newtoncotes(f, a, b, n, m):
    d = [2., 6., 8., 90., 288., 840., 17280., 28350.]
    c = [1., 1., 4., 1., 3., 7., 32., 12., 19., 75., 50., 41., 216., 27., 272., 751., 3577., 1323., 2989., 28350.,

    conderro = 0
    intgr = 0

    if n < 1 or n > 8:
        conderro += 1

    if m % n != 0:
        conderro += 2

    if conderro != 0:
        print("condERR0 = %s"%conderro)
        return None

    p = trunc(0.25*(n*(n + 2) + (n%2)))
    h = (b - a)/m

    for i in range(m+1):
        x = a + i*h
        y = f(x)
        j = p + trunc(0.5*n - abs((i%n) - 0.5*n))
        k = 1 + trunc((n - (i%n))/n) - trunc((m - (i%m))/m)
        intgr += y*c[j]*k

    intgr *= n*h/d[n-1]

    return intgr
```

Resposta da 2.3

```
In [ ]: v = lambda r: b*(1 - r/r0)**(1/7)
f = lambda r: r*v(r)

#essa é uma integral que pode ser resolvida analiticamente
```

```

Iteo = 0.408333333 #com 9 casas decimais
I = newtoncotes(f, 0, r0, 1, 10)
n = 10
#o algoritmo implementado é o mesmo da apostila e tecnicamente não tem nada errado em fazer dessa maneira,
#apesar de que deve haver um método de subdividir o intervalo e usar outras formulas de interpolação pro
#newton cotes.
#ps: em Julia e Fortran esse mesmo código roda em menos de 4s, em comparação com os mais de 2 minutos em Python
while abs(I - Iteo) > 1e-9 and n < 100000000:
    n *= 10
    I = newtoncotes(f, 0, r0, 1, n)

print("I = %.9f"%I)
print("n = %d"%n)

```

0.3801206142413862

I = 0.408333333

n = 100000000

```

In [ ]: Q = 2*np.pi*I
print("Q = %.9f"%Q)

```

Q = 2.565633999

```

In [ ]: #código de sistemas lineares
def LUdecomp(A):
    n = A.shape[0]
    m = A.shape[1]

    pivot = np.zeros(n)
    for i in range(n):
        pivot[i] = i+1

    det = 1
    for j in range(0,n-1):
        p = j
        Amax = abs(A[j,j])
        for k in range(j+1,n):
            if abs(A[k,j]) > Amax:
                Amax = abs(A[k,j])
                p = k

        if p != j:

```

```

        for k in range(0,m):
            t = A[j,k]
            A[j,k] = A[p,k]
            A[p,k] = t

        s = pivot[j]
        pivot[j] = pivot[p]
        pivot[p] = s
        det = -det

    det = det*A[j,j]
    if A[j,j] != 0:
        r = 1/A[j,j]
        for i in range(j+1,n):
            mult = A[i,j]*r
            A[i,j] = mult
            for k in range(j+1,m):
                A[i,k] = A[i,k] - mult*A[j,k]

    det = det*A[n-1,n-1]
    return (A, det, pivot)

def getL(LU):
    n = LU.shape[0]
    m = LU.shape[1]

    L = np.zeros((n,m))
    for i in range(n):
        for j in range(m):
            if i == j:
                L[i,j] = 1
            elif i > j:
                L[i,j] = LU[i,j]

    return L

def getU(LU):
    n = LU.shape[0]
    m = LU.shape[1]

    U = np.zeros((n,m))

```

```

    for i in range(n):
        for j in range(m):
            if i <= j:
                U[i,j] = LU[i,j]

    return U

def solveL(L, pivot, b):
    n = L.shape[0]

    y = np.zeros(n)
    k = int(pivot[0])
    y[0] = b[k-1]
    for i in range(1,n):
        soma = 0
        for j in range(0,i):
            soma = soma + L[i,j]*y[j]

        k = int(pivot[i])
        #tive que fazer essa adaptação porque o algoritmo do material não é genérico o suficiente
        #para permitir que seja qualquer matrix triangular inferior
        y[i] = (b[k-1] - soma)/L[i,i]

    return y

def solveU(U, y):
    n = U.shape[0]
    x = np.zeros(n)

    x[n-1] = y[n-1]/U[n-1,n-1]
    for i in reversed(range(0,n-1)):
        soma = 0
        for j in range(i+1,n):
            soma = soma + U[i,j]*x[j]

        x[i] = (y[i] - soma)/U[i,i]

    return x

def solveLU(A, b):
    LU, det, pivot = LUdecomp(A)

```

```

L = getL(LU)
U = getU(LU)
y = solveL(L, pivot, b)
x = solveU(U, y)
return x

```

```

def choleskydecomp(A):
    n = A.shape[0]

```

```

    conderro = 0
    det = 1
    r = 1
    t = 1
    soma = 0

```

```

    for j in range(0,n):
        soma = 0
        for k in range(0,j):
            soma = soma + A[j,k]**2

```

```

        t = A[j,j] - soma
        if t > 0:
            A[j,j] = math.sqrt(t)
            r = 1/A[j,j]
            det = det*t

```

```

        else:
            conderro = 1
            print("Erro: matriz não definida positiva")
            return (A, conderro, det)

```

```

    for i in range(j+1,n):
        soma = 0
        for k in range(0,j):
            soma = soma + A[i,k]*A[j,k]

```

```

        A[i,j] = (A[i,j] - soma)*r
        #algoritmo no material retornava uma matriz que não era triangular inferior (ele não mexia nos elementos
        A[j,i] = 0

```

```

    return (A, conderro, det)

```

```

def solvecholesky(A, b):

```

```

G, conderro, det = choleskydecomp(A)
#cholesky não faz pivoteamento, pivot vai ser só [1,2,3]
pivot = np.arange(1, A.shape[0]+1)
y = solveL(G, pivot, b)
x = solveU(G.T, y)
return x

```

```

def solvegauss_seidel(A, b, tol, itermax):
    n = A.shape[0]
    x = np.zeros(n)
    v = np.zeros(n)
    for i in range(0,n):
        r = 1/A[i,i]
        for j in range(0,n):
            if i != j:
                A[i,j] = A[i,j]*r

        #A[i,i] = 0
        b[i] = b[i]*r
        x[i] = b[i]

    iter = 0
    normarel = 1
    while iter <= itermax:
        iter = iter + 1
        for i in range(0,n):
            soma = 0
            for j in range(0,n):
                if i != j:
                    soma = soma + A[i,j]*x[j]

            v[i] = x[i]
            x[i] = b[i] - soma

        normanum = 0
        normaden = 0
        for i in range(0,n):
            t = abs(v[i] - x[i])
            normanum = normanum + t**2
            normaden = normaden + abs(v[i])**2

```

```

        normarel = math.sqrt(normanum/normaden)
        if normarel <= tol:
            conderro = 0
        else:
            conderro = 1

    return x, normarel, conderro

```

```

In [ ]: #código de formatação de matrizes
def formatacaough(A):
    for i in range(A.shape[0]):
        if len(A.shape) == 2:
            for j in range(A.shape[1]):
                print("[{0:^9}]" .format("%.3g" % A[i,j]), end='')
            else:
                print("[{0:^9}]" .format("%.3g" % A[i]), end='')

    print()

```

```

In [ ]: #código de ajuste funcional
#não tem um algoritmo de ajuste funcional explícito na apostila, mas utilizei o que foi descrito no moodle
def discretenorm(u, v):
    if len(u) != len(v):
        raise ValueError("Vectors must have the same length")

    return sum([u[i]*v[i] for i in range(len(u))])

def discretepolynfit(x, y, n):
    M = np.zeros((n+1,n+1))
    b = np.zeros(n+1)
    for i in range(n+1):
        for j in range(n+1):
            M[i,j] = discretenorm(x**i, x**j)

    b[i] = discretenorm(y, x**i)

    #talvez tenha outros métodos mais eficientes de resolver esse sistema, mas como o LU e gauss compacto
    #ta funcionando bem aqui eu vou manter
    return solveLU(M, b)

```

```
def discreteexpfit(x, y):
    M = np.zeros((2,2))
    b = np.zeros(2)
    for i in range(2):
        for j in range(2):
            M[i,j] = discretenorm(x**i, x**j)

            b[i] = discretenorm(np.log(y), x**i)

    return solveLU(M, b)
```

Temos a tabela de dados:

```
In [ ]: #precisa ter a célula onde isso é apresentado rodada
print(rvals)
print(vrvals)
print(rvrvals)
```

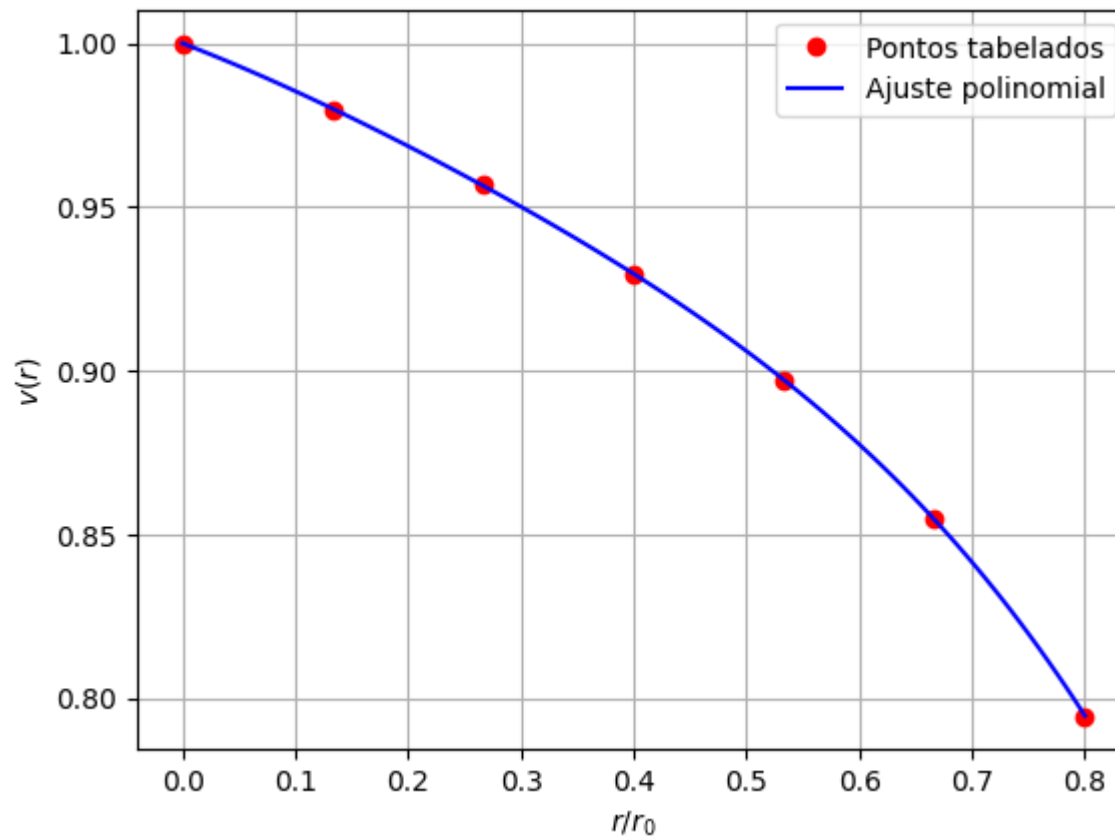
```
[0.000 0.133 0.267 0.400 0.533 0.667 0.800 0.850 0.900 0.950 1.000]
[1.000 0.980 0.957 0.930 0.897 0.855 0.795 0.763 0.720 0.652 0.000]
[0.000 0.131 0.255 0.372 0.478 0.570 0.636 0.648 0.648 0.619 0.000]
```

Vou fazer um ajuste de MMC de ordem 4 no primeiro subintervalo $([0, 0.8r_0])$ e um ajuste logaritmo no segundo subintervalo $([0.8r_0, r_0])$:

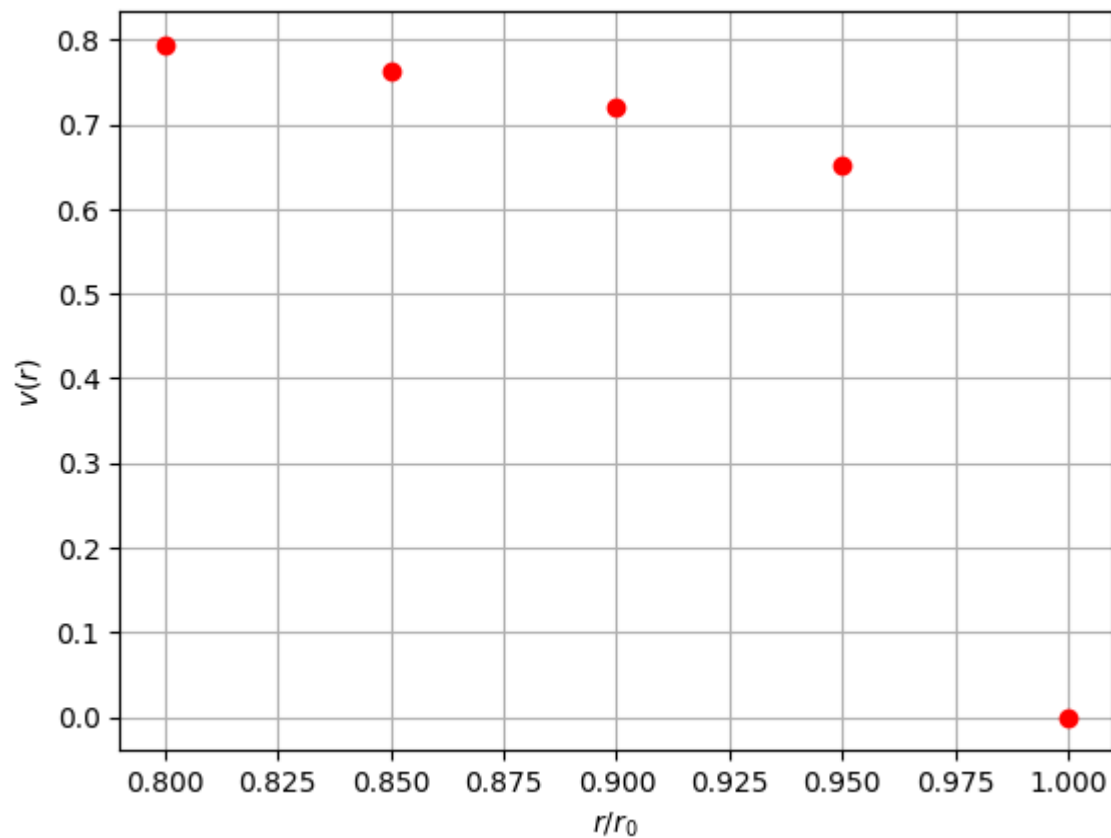
```
In [ ]: interval1 = rvals[0:7]
interval2 = rvals[6:12]
vrvals1 = vrvals[0:7]
vrvals2 = vrvals[6:12]
```

```
In [ ]: n1 = 4
coeff1 = discretepolynfit(interval1, vrvals1, n1)
ffourth1 = lambda x: sum([coeff1[i]*x**i for i in range(n1+1)])
xquad1 = np.linspace(interval1[0], interval1[-1], 100)
plt.plot(interval1, vrvals1, "ro")
plt.plot(xquad1, ffourth1(xquad1), "b-")
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
plt.legend(["Pontos tabelados", "Ajuste polinomial"])
plt.grid()
```

```
plt.show()
```



```
In [ ]: plt.plot(interval2, vrvals2, "ro", label="$v(r)$")
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
plt.grid()
plt.show()
```



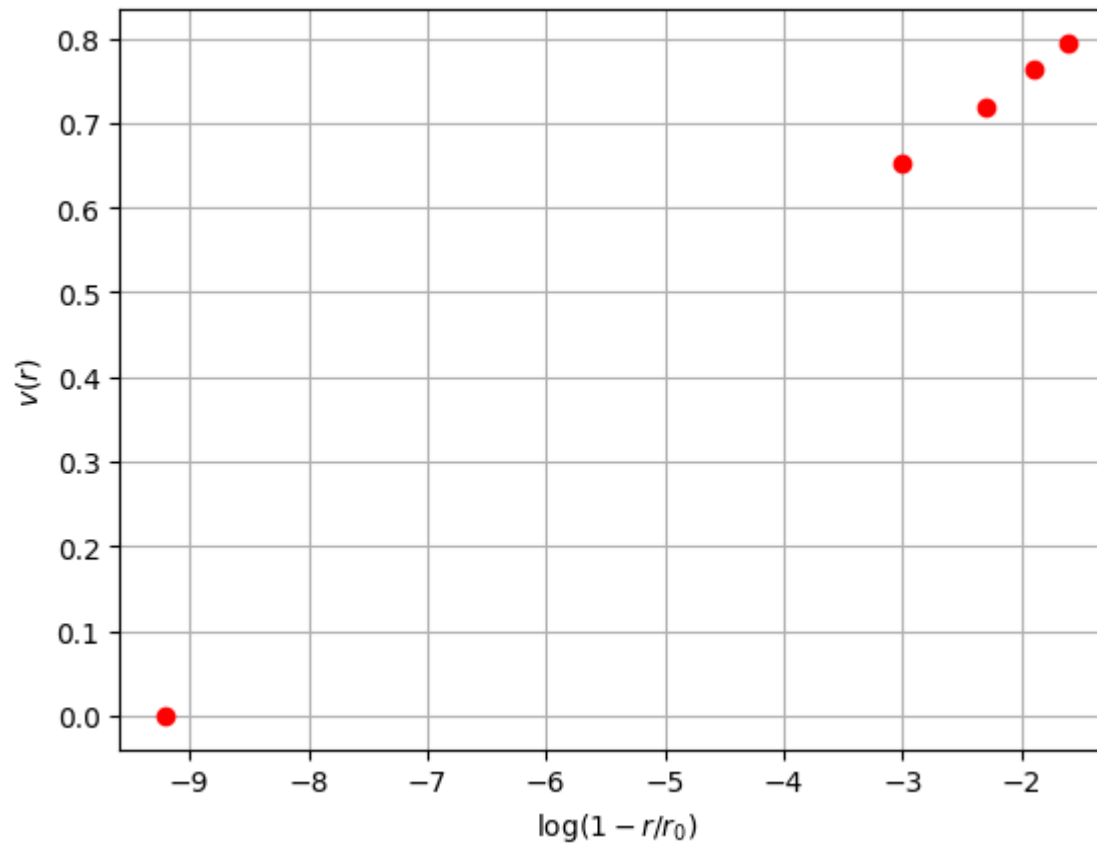
Vou tentar linearizar esse gráfico supondo a seguinte forma funcional de $v(r)$:

$$v(r) = a_1 \ln(1 - r) + a_2, \quad (11)$$

onde $r = r/r_0$. Conseguimos a partir de uma função do tipo imitar a queda súbita que aparece em $r = 1$ e, apesar de haver uma indeterminação no ponto em que o argumento do logaritmo vai a zero, ainda conseguiremos usar essa forma para aproximar a integral desejada (ainda é possível integrar uma função do tipo).

```
In [ ]: x1 = interval2
# podemos redefinir x para que seja menor que esse log ou então mudar o 1 dentro do argumento do log
#x[-1] = 0.9999
newx1 = np.log(1.0000001 - x1)
```

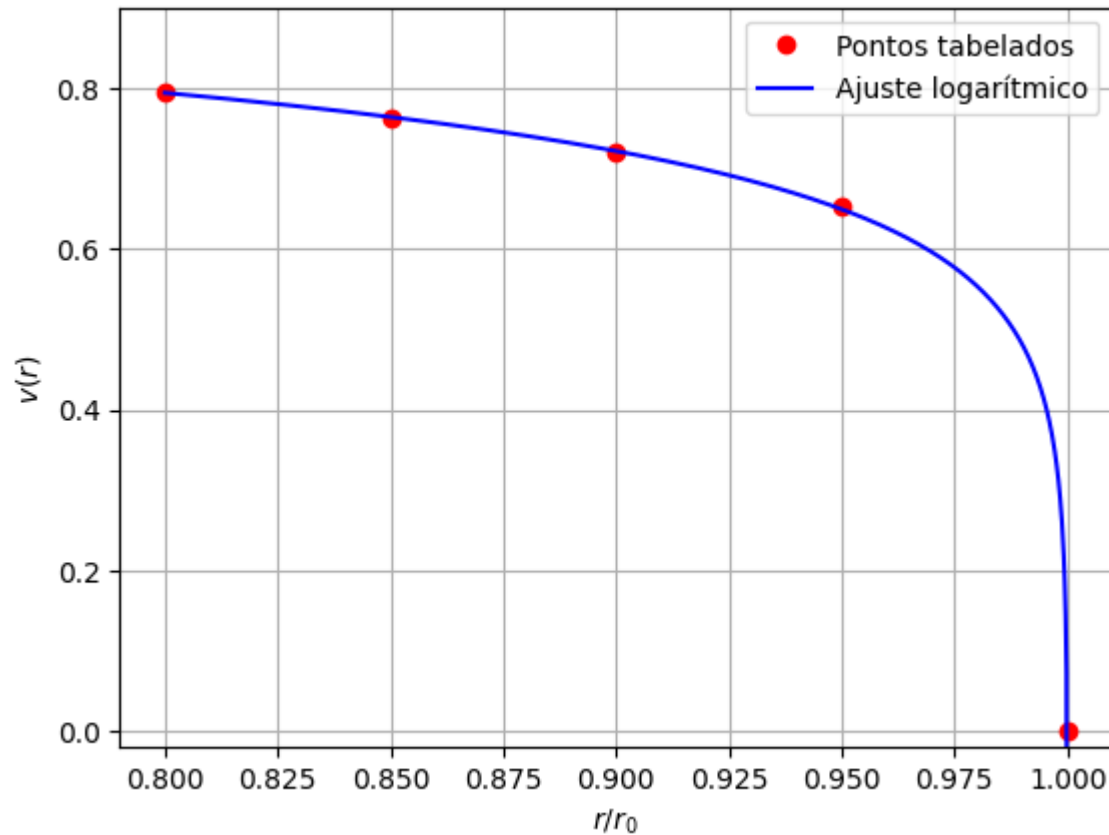
```
plt.plot(newx1, vrvals2, 'ro')
plt.xlabel("$\log(1 - r/r_0)$")
plt.ylabel("$v(r)$")
plt.grid()
plt.show()
```



```
In [ ]: coeff2 = discretepolyfit(newx1, vrvals2, 1)
flog1 = lambda x: coeff2[0] + coeff2[1]*np.log(1.0000001 - x)
xlog1 = np.linspace(0.8, 1.0, 1000)

plt.plot(interval2, vrvals2, "ro", label="$v(r)$")
plt.plot(xlog1, flog1(xlog1), "b-", label="Ajuste linear")
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
```

```
plt.ylim(-0.02, 0.9)
plt.legend(["Pontos tabelados", "Ajuste logarítmico"])
plt.grid()
plt.show()
```

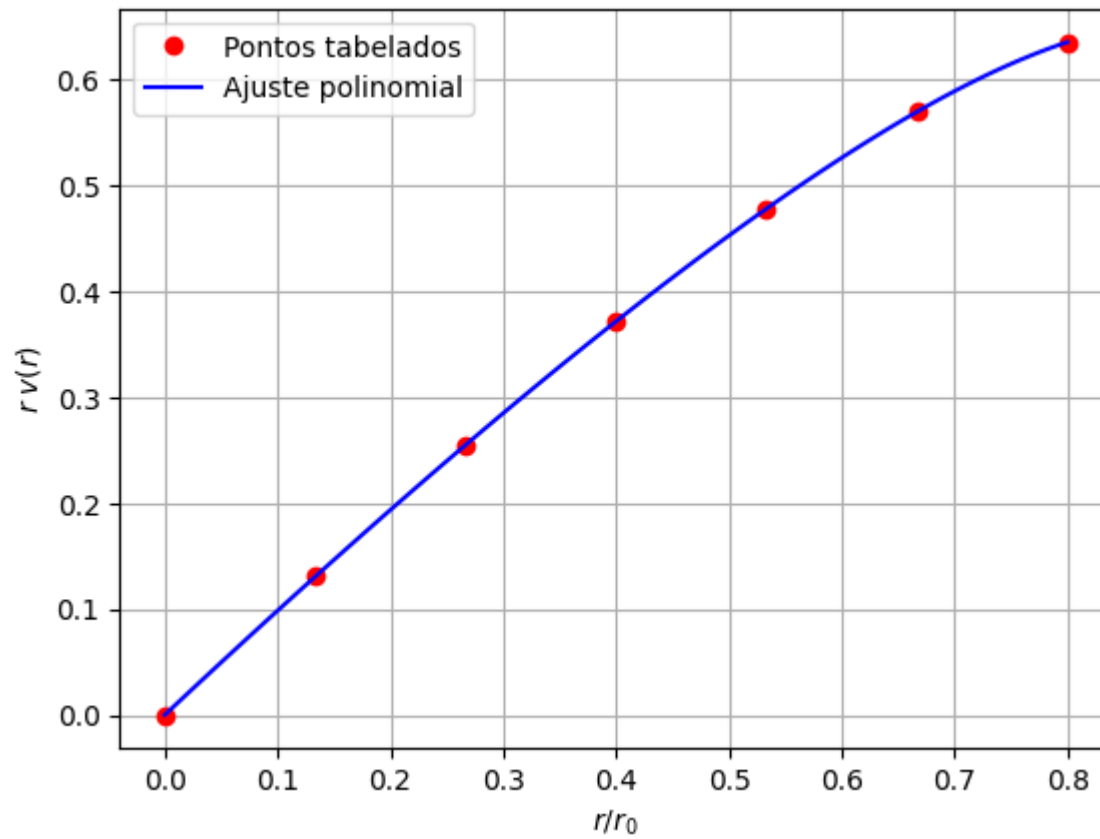


Vamos agora ajustar a segunda função (a que queremos integrar mesmo) e depois comparar tudo com o que obtemos diretamente da função desejada.

```
In [ ]: interval3 = rvals[0:7]
interval4 = rvals[6:12]
rvrvals1 = rvrvals[0:7]
rvrvals2 = rvrvals[6:12]
```

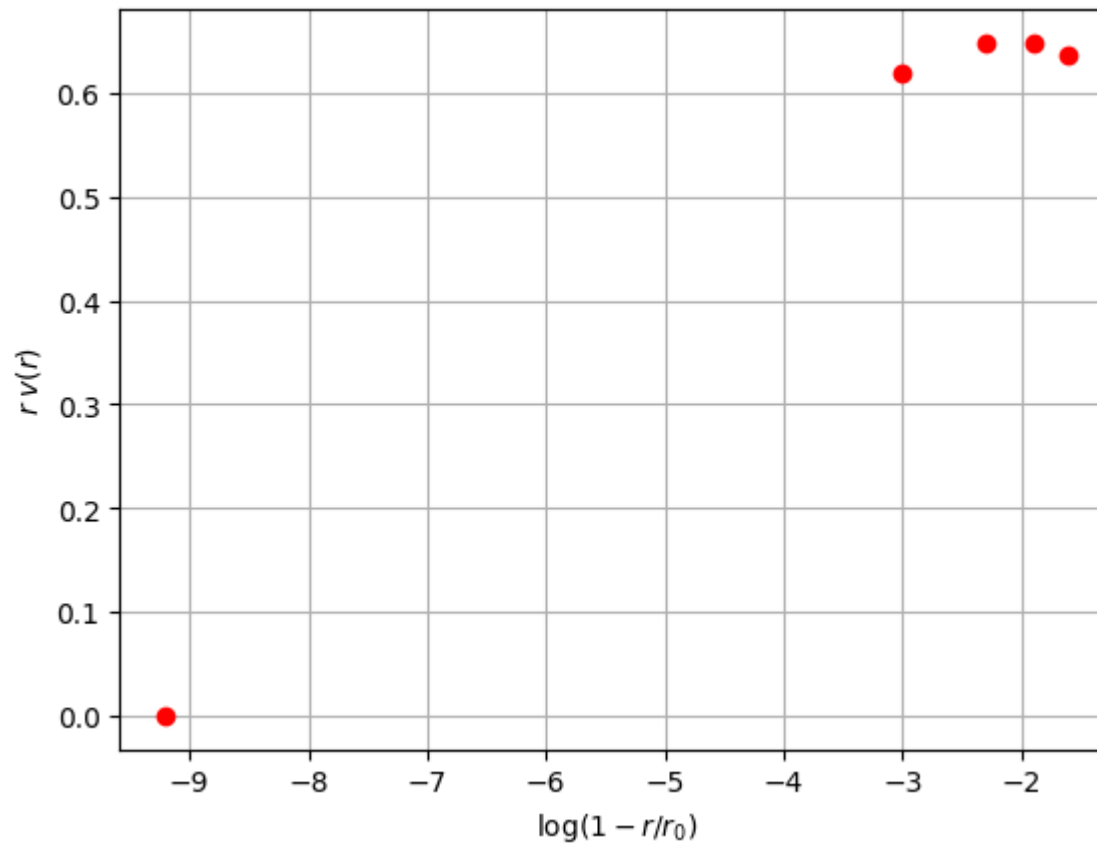
Importante notar que mudei de um ajuste de polinômio de grau 4 para um polinômio de grau 6 no primeiro subintervalo pois obtive um melhor ajuste assim.

```
In [ ]: n3 = 6
coeff3 = discretepolynfit(interval3, rvrvsl1, n3)
ffourth2 = lambda x: sum([coeff3[i]*x**i for i in range(n3+1)])
xquad2 = np.linspace(interval3[0], interval3[-1], 100)
plt.plot(interval3, rvrvsl1, "ro")
plt.plot(xquad2, ffourth2(xquad2), "b-")
plt.xlabel("$r/r_0$")
plt.ylabel("$r \setminus v(r)$")
plt.legend(["Pontos tabelados", "Ajuste polinomial"])
plt.grid()
plt.show()
```



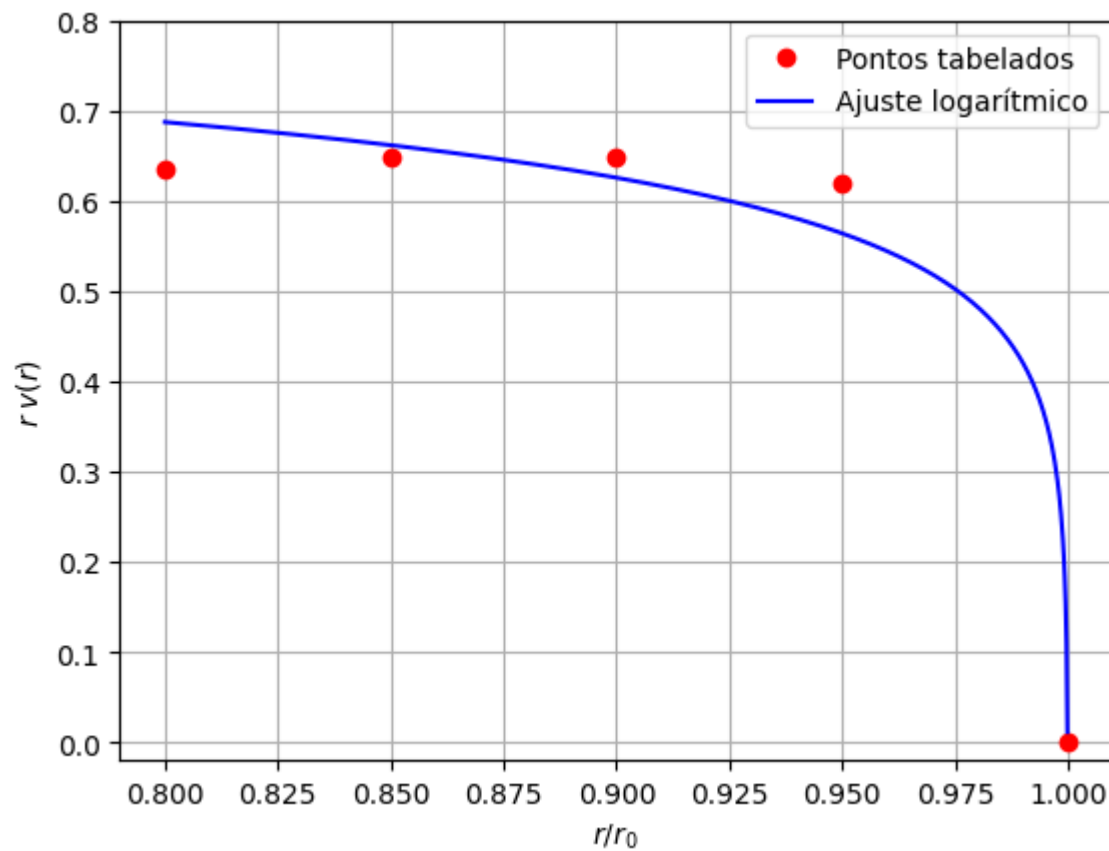
```
In [ ]: x2 = interval4
xnew2 = np.log(1.0000001 - x2)

plt.plot(xnew2, rvrvs2, 'ro')
plt.xlabel("$\log(1 - r/r_0)$")
plt.ylabel("$r \ ; \ v(r)$")
plt.grid()
plt.show()
```



Dessa vez não parece estar tão bem linearizado, mas deve bastar.

```
In [ ]: coeff4 = discretepolynfit(xnew2, rvrvs2, 1)
flog2 = lambda x: coeff4[0] + coeff4[1]*np.log(1.0000001 - x)
xlog2 = np.linspace(interval4[0], interval4[-1], 1000)
plt.plot(interval4, rvrvs2, "ro", label="$r \setminus; v(r)$")
plt.plot(xlog2, flog2(xlog2), "b-", label="Ajuste linear")
plt.xlabel("$r/r_0$")
plt.ylabel("$r \setminus; v(r)$")
plt.ylim(-0.02, 0.8)
plt.legend(["Pontos tabelados", "Ajuste logarítmico"])
plt.grid()
plt.show()
```



Juntando e colocando em um mesmo gráfico e comparando com as funções originais:

```
In [ ]: print(coeff1)
        print(coeff2)
        print(coeff3)
        print(coeff4)

[1.000 -0.132 -0.167 0.269 -0.319]
[0.962 0.104]
[-0.000 1.002 -0.186 0.214 -0.856 1.139 -0.684]
[0.831 0.089]
```

Resposta da 3.1

Encontramos os parâmetros para cada respectivo ajuste:

$$v(x = r/r_0) = \begin{cases} 1 - 0.132x - 0.167x^2 + 0.269x^3 - 0.319x^4, & \text{se } 0 < x \leq 0.8 \\ 0.962 \ln(1 - x) + 0.104, & \text{se } 0.8 < x < 1, \end{cases} \quad (12)$$

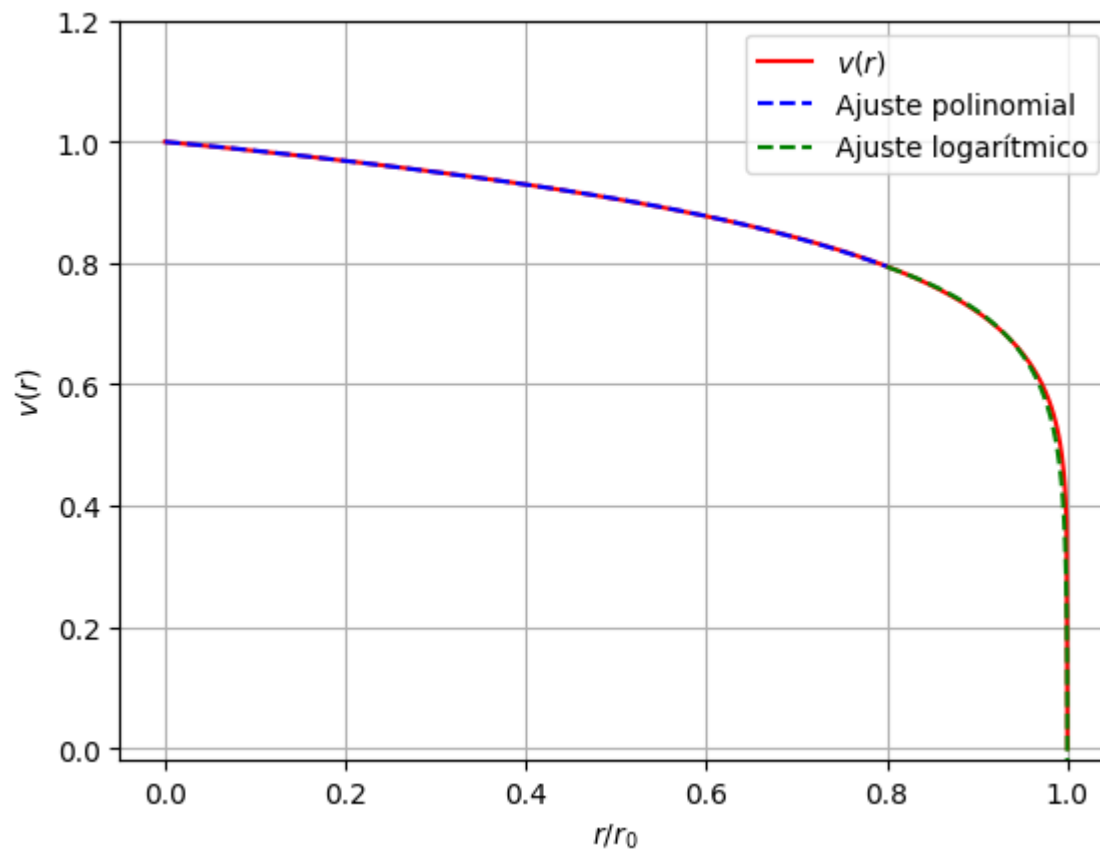
$$rv(x = r/r_0) = \begin{cases} 1.002x - 0.186x^2 + 0.214x^3 - 0.856x^4 + 1.139x^5 - 0.684x^6, & \text{se } 0 < x \leq 0.8 \\ 0.089 \ln(1 - x) + 0.831, & \text{se } 0.8 < x < 1. \end{cases} \quad (13)$$

```
In [ ]: v = lambda r: b*(1 - r/r0)**(1/7)
        rv = lambda r: r*v(r)
```

Resposta da 3.2

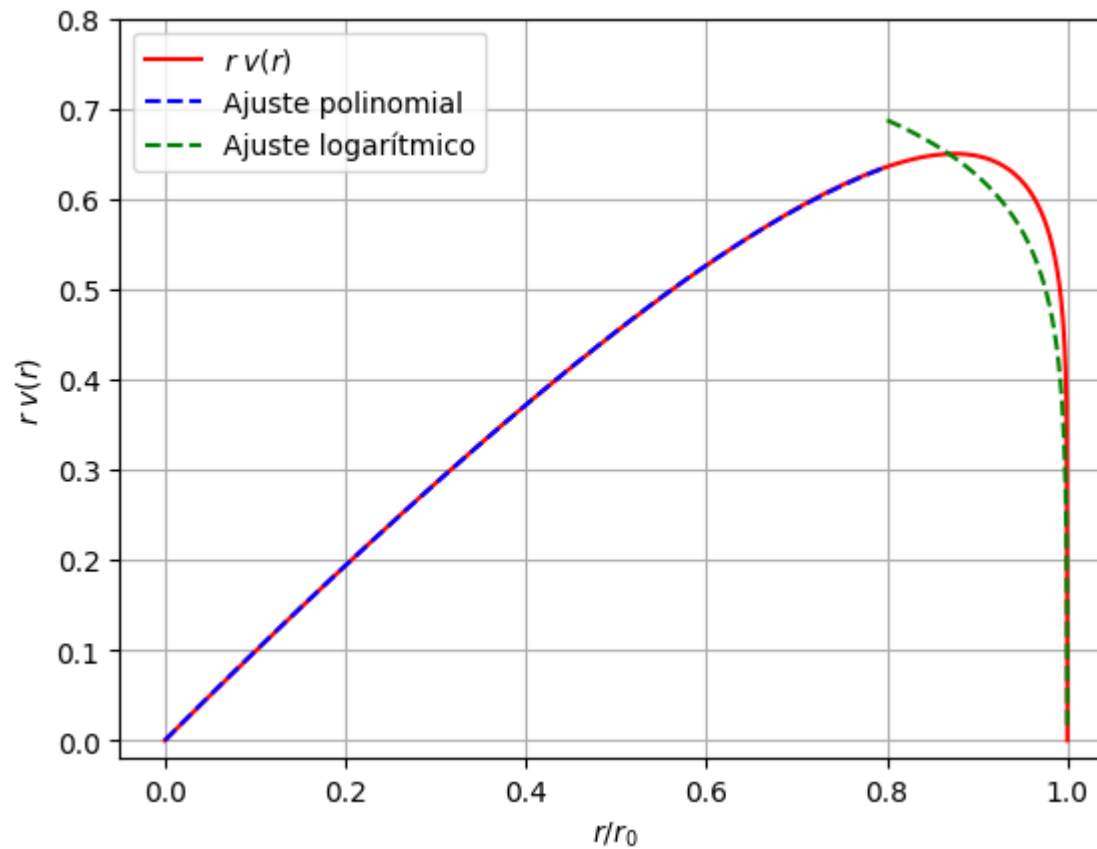
```
In [ ]: xvr = np.linspace(0, r0, 1000)

plt.plot(xvr, v(xvr), "r-", label="$v(r)$")
plt.plot(xquad1, ffourth1(xquad1), "b--")
plt.plot(xlog1, flog1(xlog1), "g--")
plt.xlabel("$r/r_0$")
plt.ylabel("$v(r)$")
plt.ylim(-0.02, 1.2)
plt.legend(["$v(r)$", "Ajuste polinomial", "Ajuste logarítmico"])
plt.grid()
plt.show()
```



```
In [ ]: xrvr = np.linspace(0, r0, 1000)

plt.plot(xrvr, rv(xrvr), "r-", label="$r \ ; \ v(r)$")
plt.plot(xquad2, ffourth2(xquad2), "b--")
plt.plot(xlog2, flog2(xlog2), "g--")
plt.xlabel("$r/r_0$")
plt.ylabel("$r \ ; \ v(r)$")
plt.ylim(-0.02, 0.8)
plt.legend(["$r \ ; \ v(r)$", "Ajuste polinomial", "Ajuste logarítmico"])
plt.grid()
plt.show()
```



Eu até tentei ajustar um polinômio e também uma função exponencial no segundo subintervalo, mas por algum motivo eu ainda estava obtendo uns ajustes não muito bons (mesmo pra primeira função $v(r)$). No fim o ajuste da primeira função ficou bem bom, o da segunda acho que daria para melhorar retirando mais alguns pontos e podendo mudar os subintervalos, mas acho que não posso fazer isso aqui.

Resposta da 3.3

A integral é:

$$Q = 2\pi I, \quad (14)$$

onde

$$I = \int_0^{r_0} r v(r) dr. \quad (15)$$

Como temos duas funções ajustadas, quebramos o intervalo:

$$I \approx \int_0^{0.8r_0} (1.002x - 0.186x^2 + 0.214x^3 - 0.856x^4 + 1.139x^5 - 0.684x^6) dr + \int_{0.8r_0}^{r_0} [0.089 \ln(1-x) + 0.831] dr, \quad x = r/r_0$$

São integrais resolvíveis analiticamente. Obtemos

$$I \approx 0.403734 \quad (17)$$

em comparação com o resultado que sabemos

$$I = 0.4083333... \quad (18)$$

Novamente, a aproximação polinomial resultou em duas casas decimais de precisão enquanto a integral com o ajuste logarítmico resultou em apenas uma casa decimal de precisão (cálculos feitos no Wolfram Alpha).

A equação diferencial encontrada é:

$$\begin{cases} 2y' = 1 + \frac{y}{2x}, x \in [1, 2] \\ y(1) = 2, (h = 0,05) \end{cases} \quad (19)$$

Essa EDO tem solução analítica

$$y(x) = \frac{2}{3}(2x^{1/4} + x) \quad (20)$$

```
In [ ]: yf = lambda x: 2.0*(2*x**(1/4) + x)/3.0
```

```
In [ ]: #código feito para solução de edos  
#não tem nenhum algoritmo explícito na apostila, mas utilizei o texto nela como base
```

```

def euler(f, x0, y0, h, n):
    #floor não funciona muito bem por causa de divisões estranhas de floats, ex: 0.3/0.1 = 2.9999999999999996
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        x[i] = x[i-1] + h
        y[i] = y[i-1] + h*f(x[i-1],y[i-1])

    return (x,y)

def heun(f, x0, y0, h, n):
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        K1 = f(x[i-1], y[i-1])
        K2 = f(x[i-1] + h/3.0, y[i-1] + h*K1/3.0)
        K3 = f(x[i-1] + 2.0*h/3.0, y[i-1] + 2.0*h*K2/3.0)
        y[i] = y[i-1] + h*(K1 + 3.0*K3)/4.0
        x[i] = x[i-1] + h

    return (x,y)

#é facilmente generalizado para sistemas de equações diferenciais
def rk4(f, x0, y0, h, n):
    x = np.zeros(n+1)
    y = np.zeros(n+1)
    x[0] = x0
    y[0] = y0
    for i in range(1,n+1):
        K1 = f(x[i-1], y[i-1])
        K2 = f(x[i-1] + h/2.0, y[i-1] + h*K1/2.0)
        K3 = f(x[i-1] + h/2.0, y[i-1] + h*K2/2.0)
        K4 = f(x[i-1] + h, y[i-1] + h*K3)
        y[i] = y[i-1] + h*(K1 + 2.0*K2 + 2.0*K3 + K4)/6.0
        x[i] = x[i-1] + h

    return (x,y)

```

```
In [ ]: n = 20
x0 = 1.0
y0 = 2.0
f = lambda x,y: 0.5 + y/(4.0*x)
h = 0.05
x, y = heun(f, x0, y0, h, n)
```

```
In [ ]: print(x)
print(y)
print(yf(x))
```

```
[1.000 1.050 1.100 1.150 1.200 1.250 1.300 1.350 1.400 1.450 1.500 1.550
 1.600 1.650 1.700 1.750 1.800 1.850 1.900 1.950 2.000]
[2.000 2.050 2.099 2.147 2.196 2.243 2.290 2.337 2.384 2.430 2.476 2.521
 2.566 2.611 2.656 2.700 2.744 2.788 2.832 2.876 2.919]
[2.000 2.050 2.099 2.147 2.196 2.243 2.290 2.337 2.384 2.430 2.476 2.521
 2.566 2.611 2.656 2.700 2.744 2.788 2.832 2.876 2.919]
```

Resposta da 3.4

Tenho uma restrição da avaliação de que o passo h deve ser 0.5 e também que x está no intervalo $[0, 1]$. Essa solução se comporta quase que como uma reta de modo que até o método de Euler, quando testado vai dar um resultado muito preciso. Preciso fornecer de 10 em 10 valores, porém, como o passo e o intervalo são fixos eu tenho somente 21 pontos (contando como o inicial). Vou fornecer uma tabela com a solução numérica comparada com a exata de 10 em 10 pontos e outra de 5 em 5 pontos.

```
In [ ]: print("n \t x_(n+1) \t y_(n+1) \t y_E \t\t erro absoluto")
for i in range(1,n+1, 10):
    print("%d \t %.4f \t %.4f \t %.4f \t %.4f"%(i, x[i], y[i], yf(x[i]), abs(y[i] - yf(x[i]))))
```

n	x_(n+1)	y_(n+1)	y_E	erro absoluto
1	1.0500	2.0497	2.0497	0.0000
11	1.5500	2.5211	2.5211	0.0000

Agora começando da solução y_0

```
In [ ]: print("n \t x_(n+1) \t y_(n+1) \t y_E \t\t erro absoluto")
for i in range(0,n+1, 10):
```

```
print("%d \t %.4f \t %.4f \t %.4f \t %.4f"%(i, x[i], y[i], yf(x[i]), abs(y[i] - yf(x[i]))))
```

n	x_(n+1)	y_(n+1)	y_E	erro absoluto
0	1.0000	2.0000	2.0000	0.0000
10	1.5000	2.4756	2.4756	0.0000
20	2.0000	2.9189	2.9189	0.0000

Agora de 5 em 5:

```
In [ ]: print("n \t x_(n+1) \t y_(n+1) \t y_E \t\t erro absoluto")
        for i in range(1,n+1, 5):
            print("%d \t %.4f \t %.4f \t %.4f \t %.4f"%(i, x[i], y[i], yf(x[i]), abs(y[i] - yf(x[i]))))
```

n	x_(n+1)	y_(n+1)	y_E	erro absoluto
1	1.0500	2.0497	2.0497	0.0000
6	1.3000	2.2904	2.2904	0.0000
11	1.5500	2.5211	2.5211	0.0000
16	1.8000	2.7444	2.7444	0.0000

```
In [ ]: print("n \t x_(n+1) \t y_(n+1) \t y_E \t\t erro absoluto")
        for i in range(0,n+1, 5):
            print("%d \t %.4f \t %.4f \t %.4f \t %.4f"%(i, x[i], y[i], yf(x[i]), abs(y[i] - yf(x[i]))))
```

n	x_(n+1)	y_(n+1)	y_E	erro absoluto
0	1.0000	2.0000	2.0000	0.0000
5	1.2500	2.2432	2.2432	0.0000
10	1.5000	2.4756	2.4756	0.0000
15	1.7500	2.7002	2.7002	0.0000
20	2.0000	2.9189	2.9189	0.0000

```
In [ ]: xmesh = np.linspace(min(x), max(x), 100)
        ymesh = np.linspace(min(y), max(y), 100)

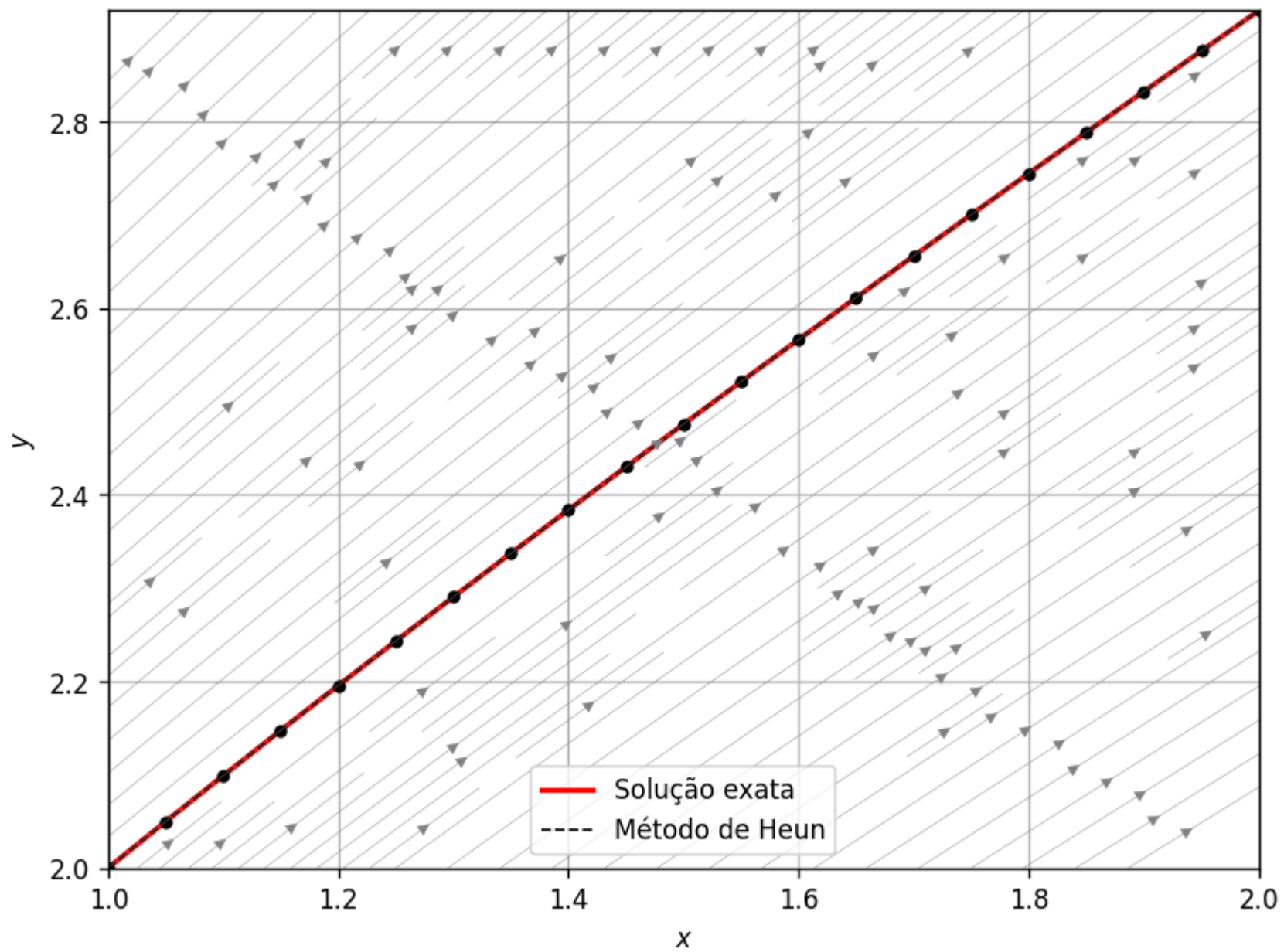
        Xl, Yl = np.meshgrid(xmesh, ymesh)
        P = Xl/Xl
        Q = 0.5 + Yl/(4.0*Xl)

        fig = plt.figure(figsize=(8, 6), dpi=120)

        plt.plot(x, yf(x), "r-", label="Solução exata", lw=2)
        plt.plot(x, y, "k--", label="Método de Heun", lw=1)
        plt.plot(x, y, "ko", ms=4)
```

```
plt.streamplot(Xl, Yl, P, Q, density=1.5, color="gray", linewidth=0.25)

plt.xlabel("$x$")
plt.ylabel("$y$")
plt.legend(["Solução exata", "Método de Heun"])
plt.grid()
plt.show()
```



In []: